Project- High Level Design

On

AI Manufacturing Platform

Course Name: DevOps

Datagami Skill Based Course

Student Names & Enrolment Numbers:

Sr no	Student Name	Enrolment Number
1	Aaditya Shrivastava	EN22CS301007
2	Aakansha Jaiswal	EN22CS301011
3	Akanksha Sharma	EN22CS301080
4	Akshita Bhawsar	EN22CS301098
5	Abhishek Malviya	EN22CS301034

Group Name: 12 D1

Project Number: DO-12

Industry Mentor Name: Aashruti Shah

University Mentor Name: Prof. Dr. Ritesh Joshi

Academic Year: 2026

Table of Contents

1. Introduction
    1.1. Scope of the document
    1.2. Intended Audience
    1.3. System Overview

2. System Design
    2.1. Application Design
    2.2. Process Flow
    2.3. Information Flow
    2.4. Components Design
    2.5. Key Design Considerations
    2.6. API Catalogue

3. Data Design
    3.1. Data Model
    3.2. Data Access Mechanism
    3.3. Data Retention Policies
    3.4. Data Migration

4. Interfaces
5. State and Session Management
6. Caching

7. Non-Functional Requirements
    7.1. Security Aspects
    7.2. Performance Aspects

8. References

INTRODUCTION

1.1 Scope of the document

The scope of this document is to define the high-level design and implementation approach for the AI Manufacturing Platform, developed as part of the DevOps Skill Based Course at Medicaps University in collaboration with Datagami. This document outlines the overall system architecture, tools, workflows, and processes required to build and deploy an AI-powered manufacturing solution that leverages both Generative AI and Agentic AI capabilities.

The platform is designed to accept natural language input describing a manufacturing idea and automatically generate a structured technical description, a visual prototype image, and a detailed procurement or research report. The document covers the architecture of key modules including the Generative AI component for text and image generation, the Multi-Agent system for research and report writing, and the containerized deployment infrastructure using Docker, Kubernetes, and CI/CD pipelines.

Security, scalability, and maintainability are addressed throughout to support production readiness and future enhancements. Overall, this document serves as a comprehensive guide for designing and implementing an efficient, AI-driven manufacturing automation platform using modern DevOps and IaC methodologies.

1.2 Intended Audience

The intended audience includes developers working on AI backend modules, DevOps engineers responsible for deployment and automation, testers validating system behavior, and faculty members or evaluators reviewing the project. Cloud architects and infrastructure engineers will benefit from understanding Docker, Kubernetes, and CI/CD deployment strategies. Project managers and technical leads can use this document for a high-level understanding of system architecture and AI module integration.

1.3 System Overview

The AI Manufacturing Platform is a modern, intelligent, and automated system leveraging Generative AI and Multi-Agent AI capabilities to simplify and automate manufacturing-related tasks. Users input a manufacturing idea in natural language and receive a structured technical description, a visual prototype image, and a detailed research or procurement report.

The system integrates a Language Model module for structured text generation, an Image Generation module for visual prototypes, and a Multi-Agent system (Research Agent + Writer Agent). Built with Python and Streamlit, it uses ChromaDB for vector-based semantic memory and JSON logs for session persistence. CI/CD pipelines ensure automated build and deployment while Kubernetes provides orchestration with rolling updates for high availability.

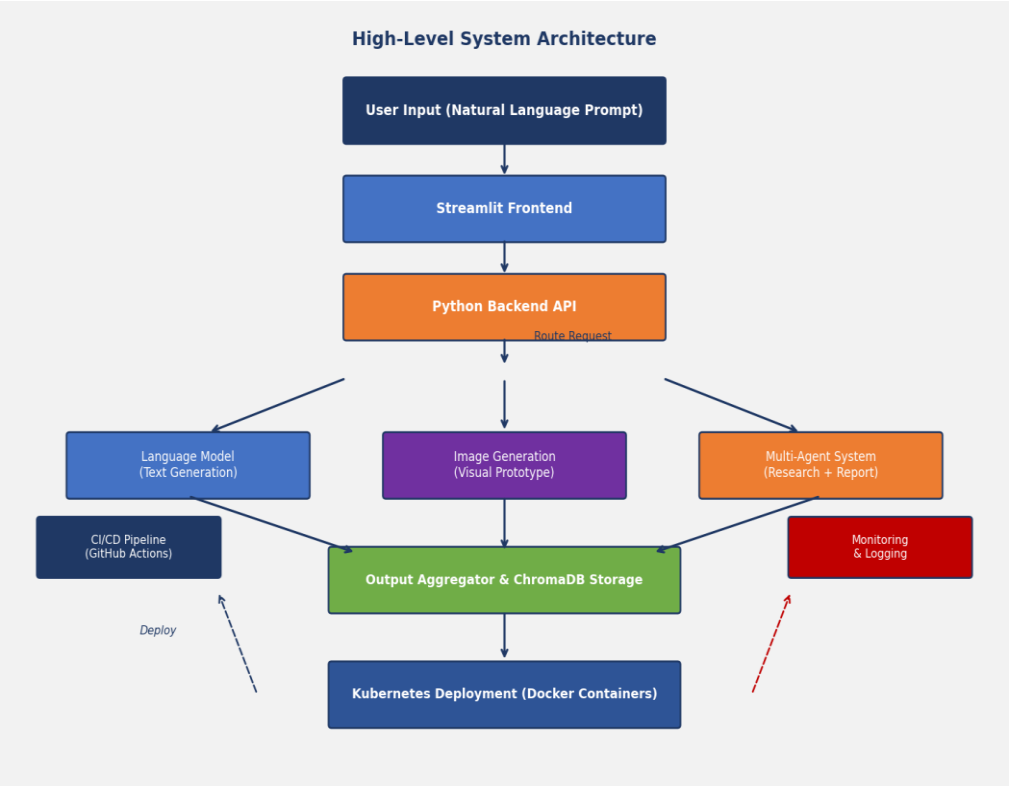

Figure 1: High-Level System Architecture – AI Manufacturing Platform

SYSTEM DESIGN

2.1 Application Design

The AI Manufacturing Platform follows a modular and containerized architecture with clearly separated layers — Frontend, Backend, AI Processing, and Storage — ensuring independent scalability and maintainability.

- The system follows a layered modular architecture separating Frontend, Backend, AI Processing, and Storage concerns for independent development and deployment.
- The Frontend Layer is built using Streamlit, providing an interactive web interface for prompt submission, output viewing, and accessing historical results.
- The Backend Layer handles all business logic and request coordination, integrating the LLM, image generation model, and multi-agent system, exposing REST APIs.
- The AI Processing Layer contains the Language Model module, Image Generation module, and Multi-Agent system (Research Agent + Writer Agent).
- The Storage Layer uses ChromaDB for vector embeddings and semantic search, and JSON-based storage for session memory and application logs.
- CI/CD pipelines using GitHub Actions automate build, test, and deployment processes triggered by code commits.
- The application is deployed and managed using Kubernetes, ensuring high availability, load balancing, rolling updates, and auto-scaling of services.
- Environment configurations and API credentials are managed securely via environment variables — never hardcoded in source.

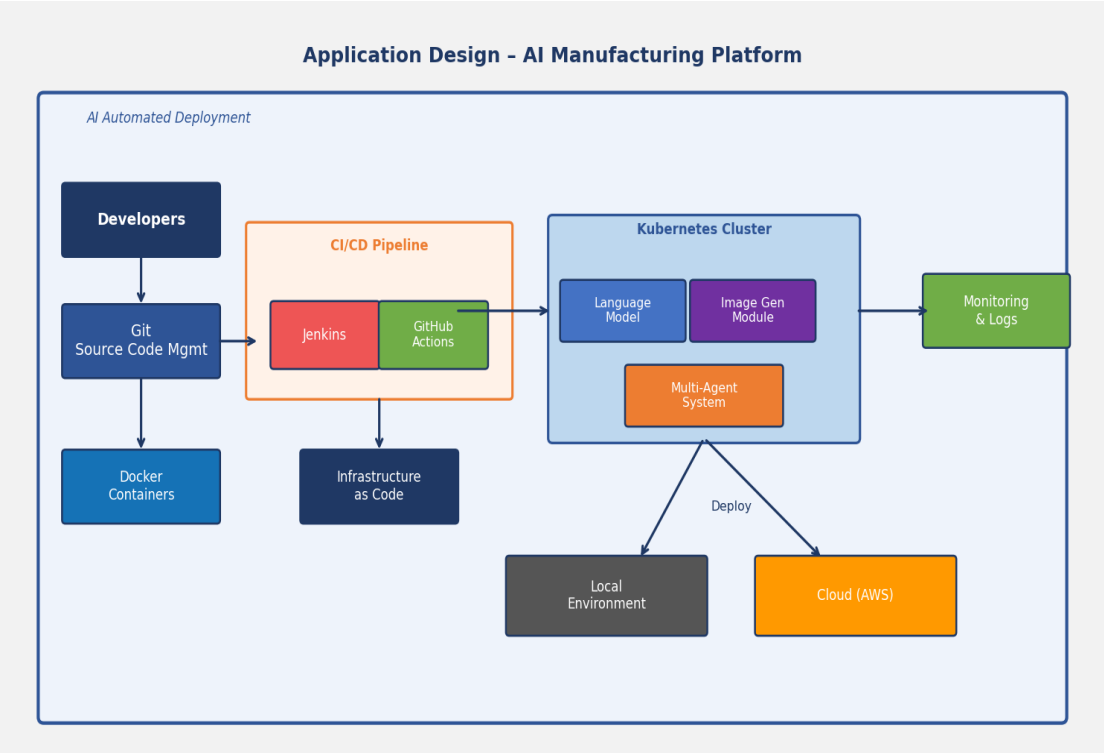

Figure 2: Application Design – AI Manufacturing Platform

2.2 Process Flow

The AI Manufacturing Platform follows a sequential and automated workflow from user input to final output delivery, with CI/CD automation and Kubernetes orchestration at every deployment stage.

- The process begins when a user enters a manufacturing query in natural language through the Streamlit web interface.
- The request is transmitted to the backend service via HTTP, where the backend determines the type of AI processing required.
- The CI/CD pipeline automates build and test stages; after passing, the application is packaged into Docker containers.
- The AI Processing Layer is triggered: the Language Model generates structured text, the Image Generation module creates a prototype, and the Multi-Agent system produces a research report.
- Generated outputs are stored in ChromaDB as vector embeddings for future semantic retrieval; logs are maintained in JSON storage.
- The Kubernetes cluster manages deployment, service discovery, load balancing, rolling updates, and auto-scaling.

- Post-deployment, monitoring and logging mechanisms track system performance and detect runtime issues in real time.
- On failure, the pipeline rolls back to the last stable container image, ensuring reliability and minimal downtime.

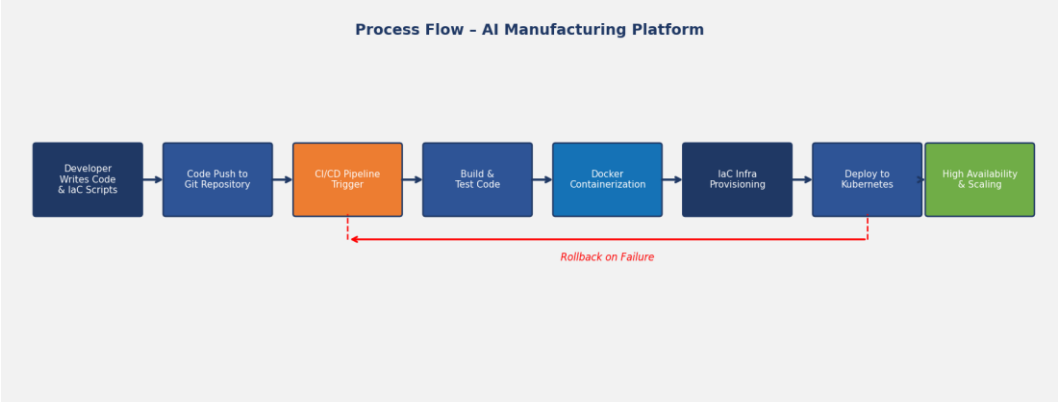

Figure 3: Process Flow – CI/CD to Kubernetes Deployment

2.3 Information Flow

The information flow is designed to be efficient, secure, and traceable from user interaction through AI processing to final output and storage.

- Users submit manufacturing prompts via the Streamlit frontend; requests route to the backend via RESTful HTTP APIs.
- The backend communicates with AI APIs for text/image generation, agent modules for research tasks, and storage systems for saving and retrieving data.
- Code commits to Git trigger automated CI/CD pipeline execution, propagating changes through build, test, and deployment stages.
- Docker containers package all application dependencies, ensuring consistent runtime environments across dev, staging, and production.
- Kubernetes-managed services communicate internally within the cluster for efficient service-to-service data exchange.
- User interaction data and AI outputs are persisted in ChromaDB (vector embeddings) and JSON logs, enabling the /history API.
- Monitoring tools capture metrics, logs, and events in real time; feedback flows back to developers and ops teams for continuous improvement.

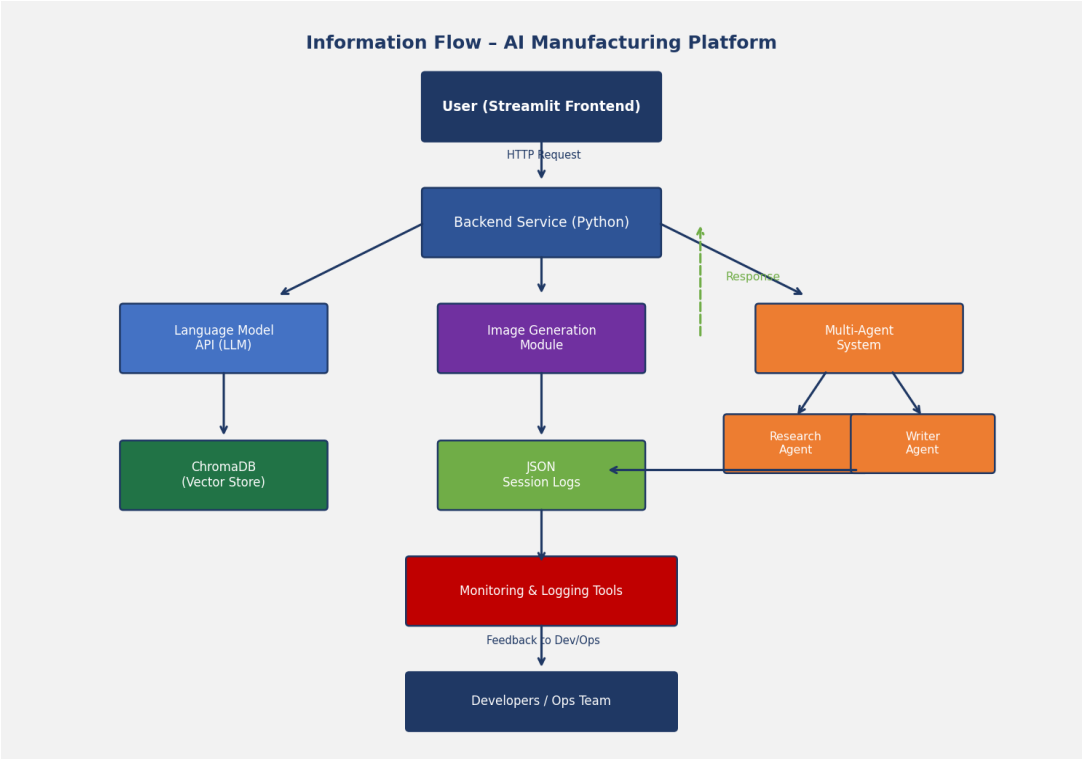

Figure 4: Information Flow – From User Input to Storage

2.4 Components Design

The platform is divided into well-defined, independently deployable components, each owning a specific responsibility in the overall architecture.

- Frontend Component: Streamlit-based UI allowing prompt submission, output display (text, image, report), and history access.
- Backend Component: Python service handling request coordination, integrating all AI modules, and exposing REST APIs.
- Language Model Module: Generates structured manufacturing descriptions, technical specifications, and optimized AI prompts.
- Image Generation Module: Produces visual prototype images of manufacturing concepts from user-provided text.
- Multi-Agent System: Research Agent gathers data; Writer Agent generates professional procurement and research reports.
- Storage Components: ChromaDB stores vector embeddings for semantic search; JSON storage maintains session logs.
- Containerization Component: Docker packages all layers for consistent, portable deployment.

- Orchestration Component: Kubernetes manages deployment, horizontal scaling, service exposure, and rolling updates.
- CI/CD Component: GitHub Actions automates the full build, test, and deployment pipeline.

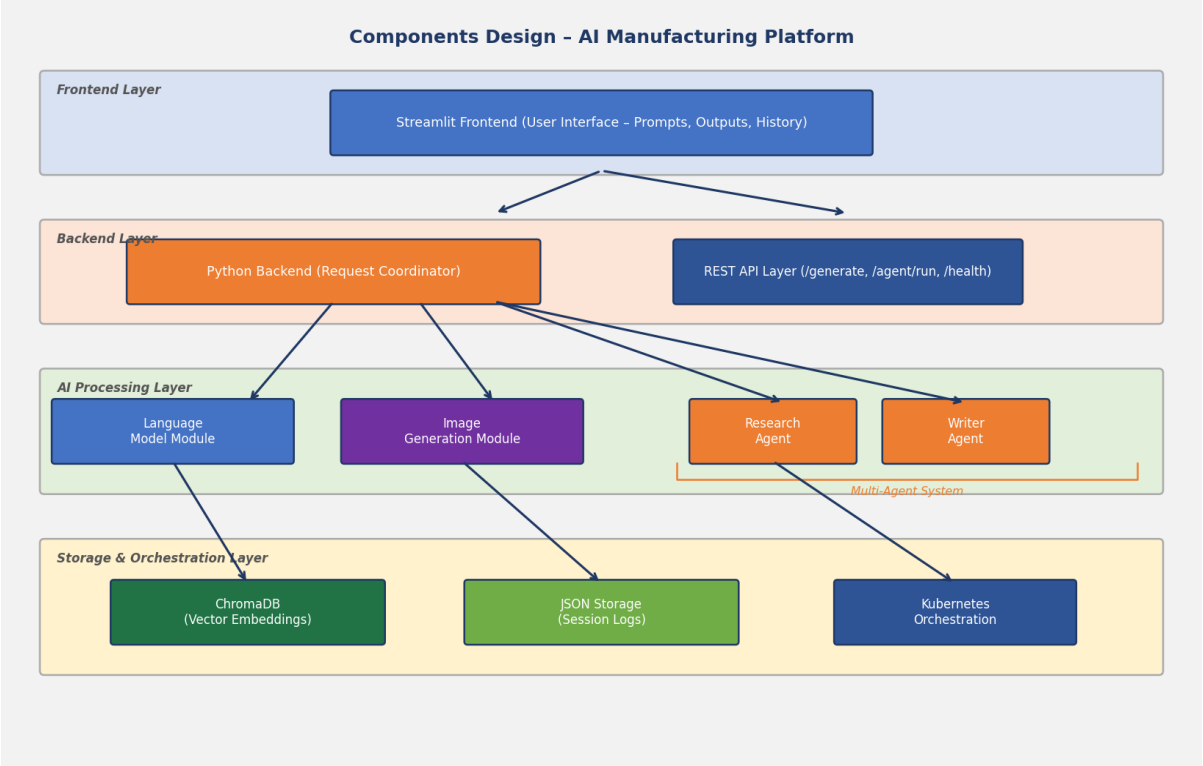

Figure 5: Components Design – Layered Architecture

2.5 Key Design Considerations

The key design considerations for the AI Manufacturing Platform focus on ensuring scalability, reliability, security, and maintainability while integrating advanced AI capabilities with robust DevOps practices.

Scalability is achieved through Kubernetes-based horizontal scaling, increasing pods based on real-time demand. High availability is maintained through rolling updates ensuring zero-downtime deployments. Security is enforced by storing all API keys through environment variables, securing inter-service communication, and scanning container images in the CI/CD pipeline. Modularity is built in so that AI models, image generators, or agent components can be independently replaced or upgraded. Clear separation of concerns and structured REST APIs make the system easy to maintain, extend, and test

2.6 API Catalogue

The API Catalogue defines RESTful APIs enabling communication between the Streamlit frontend, Python backend, and AI processing modules. APIs cover content generation, multi-agent research workflows, session history retrieval, and operational monitoring.

Endpoint	Method	Description
/generate	POST	Generates structured text description and visual prototype image from user prompt
/agent/run	POST	Executes multi-agent workflow (Research Agent + Writer Agent) to produce reports
/history	POST	Retrieves previous session results from ChromaDB and JSON storage
/health	GET	Checks if the system and all services are running correctly
/version	GET	Returns the current application version for deployment tracking

DATA DESIGN

3.1 Data Model

The data model captures core entities including Users, Manufacturing Requests, AI Outputs, Agent Reports, ChromaDB Embeddings, and Session Logs, ensuring structured management of all data generated within the platform.

- User Entity: UserID (PK), Name, Email, Phone, Role, Password — enables user-level tracking and personalization.
- Manufacturing Request Entity: RequestID (PK), UserID (FK), Prompt, Status, Timestamp — central record for each AI workflow.
- AI Output Entity: OutputID (PK), RequestID (FK), TextDescription, ImageRef, CreatedAt — links input requests to AI-generated results.
- Agent Report Entity: ReportID (PK), RequestID (FK), ResearchData, WriterReport, Timestamp — stores multi-agent generated reports.
- ChromaDB Embeddings: EmbeddingID (PK), OutputID (FK), Vector, Metadata — enables semantic similarity search.
- Session Log Entity: LogID (PK), UserID (FK), Action, Timestamp — maintains JSON-based activity history for the /history API.
- Relationships: one-to-many (one User → many Requests), one-to-one (one Request → one AI Output set), one-to-many (one Output → many Embeddings).

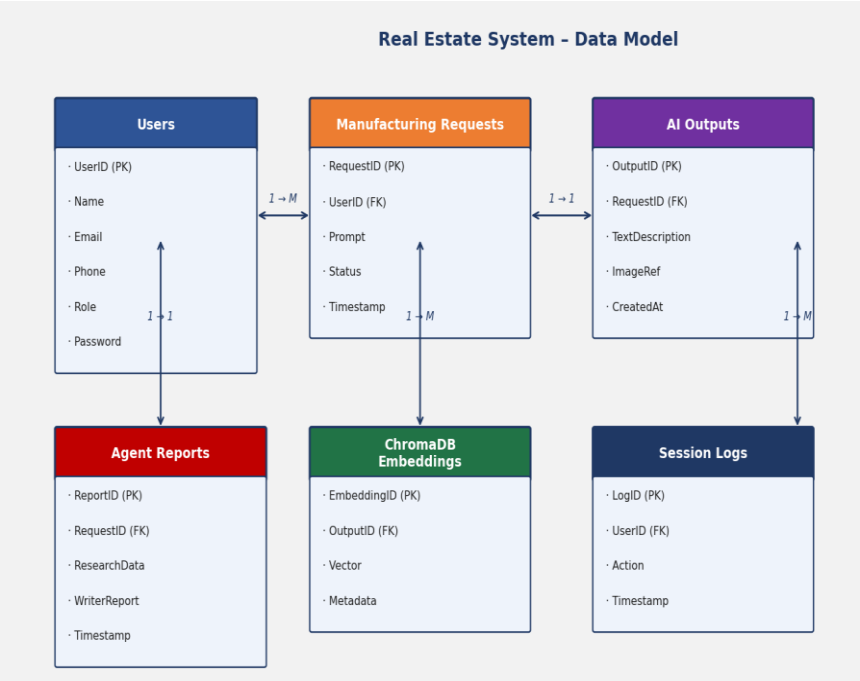

Figure 6: Data Model – Entity Relationships

3.2 Data Access Mechanism

The data access mechanism follows a layered architecture ensuring controlled, secure, and efficient access to all stored data.

- All data interactions are performed via RESTful APIs, ensuring structured and secure communication between the frontend, backend, and storage layers.
- ChromaDB queries use semantic similarity search for efficient retrieval of contextually relevant past manufacturing outputs.
- Authentication and authorization mechanisms ensure only authorized requests can access or modify stored data.
- Sensitive data such as API keys are stored using environment variables and secure configuration methods.
- Transaction-like consistency is maintained during AI output generation and storage to prevent partial failure corruption.
- Kubernetes-managed services ensure high availability and scalable access to the storage layer across multiple application instances.

3.3 Data Retention Policies

- Clear retention periods are defined for different data types: user session logs, manufacturing requests, AI outputs, and agent-generated reports.
- Critical outputs such as AI-generated descriptions, prototype images, and research reports are retained for a longer duration for user reference and auditing.
- Temporary data such as intermediate processing logs and cached session states are automatically cleared after TTL expiry.
- ChromaDB embeddings are periodically pruned when no longer actively referenced, maintaining search performance and storage efficiency.
- Regular data archiving moves older session data to cost-effective storage while keeping it accessible for historical queries.
- Automated scripts handle periodic data cleanup and log rotation, reducing storage overhead and improving system performance.

3.4 Data Migration

Data migration refers to the process of transferring existing session data, vector embeddings, and historical outputs to updated or newly provisioned infrastructure. The migration strategy includes extraction from existing JSON logs and ChromaDB collections, transformation to align with updated schema definitions, and loading into the new environment using automated CI/CD pipeline steps.

Migration scripts are version-controlled in Git and validated at each stage to prevent data corruption. Backup snapshots are taken before initiating any migration. Docker containerization ensures migration tools run consistently across environments. Post-migration validation verifies data accuracy and system functionality before cutover.

INTERFACES

The Interfaces in the AI Manufacturing Platform define how different components interact with each other and with external AI services. The system primarily uses RESTful APIs as the interface between the Streamlit frontend and the Python backend, enabling prompt submission and AI output retrieval.

Internal service-to-service communication is handled through well-defined callable interfaces within the modular backend: the Language Model module, Image Generation module, and Multi-Agent system each expose clean interfaces consumed by the backend coordinator, ensuring loose coupling and independent replaceability of AI components.

For automation and deployment, GitHub Actions acts as the interface between the code repository and the deployment environment. Docker manages container interfaces ensuring consistency across environments. Kubernetes handles the orchestration interface managing communication between services, load balancing, rolling updates, and scaling. External interfaces include HTTPS-secured API calls to language model and image generation services. All API contracts are version-controlled in Git.

STATE AND SESSION MANAGEMENT

- The system manages state in a stateless manner at the backend level, ensuring scalability and easy distribution across multiple Kubernetes pods.
- Session management uses session IDs that allow the frontend to associate user interactions with stored session data without persisting state on the server.
- User session data including past prompts and generated outputs is maintained in JSON-based storage on the backend, retrievable via the /history API.
- Backend services remain stateless, enabling seamless horizontal scaling using Kubernetes without dependency on any single server instance.
- ChromaDB provides persistent semantic memory, allowing retrieval of contextually relevant past results across separate user sessions.
- Session expiration and cleanup mechanisms automatically remove stale session logs after a defined period, ensuring storage efficiency and data privacy.

CACHING

Caching in the AI Manufacturing Platform is implemented to enhance performance, reduce latency, and minimize load on backend AI services and external API calls. The system uses ChromaDB as a semantic caching layer where previously generated outputs are stored as vector embeddings.

When a new user prompt is submitted, the system performs a semantic similarity search to check whether a sufficiently similar result already exists, returning cached output when appropriate rather than triggering a full AI processing cycle. This significantly reduces API costs and response latency for repeated or similar queries.

At the application level, caching is integrated within Docker-containerized services, ensuring consistent behavior across environments. Kubernetes-managed instances share access to ChromaDB's cached data, improving scalability and reducing redundant processing. Cache invalidation ensures outdated results are refreshed when users explicitly request new generation. Time-based TTL policies automatically clear cached data after a defined period to maintain result freshness.

NON-FUNCTIONAL REQUIREMENTS

7.1 Security Aspects

Security is a critical non-functional requirement for the AI Manufacturing Platform, ensuring protection of user data, AI-generated outputs, and system infrastructure. All external API credentials (LLM, image generation) are stored exclusively through environment variables and never embedded in source code. Communication between the Streamlit frontend and Python backend is secured using HTTPS. Container security is enforced through minimal base images and vulnerability scanning within the CI/CD pipeline.

At the infrastructure level, Kubernetes namespace isolation, Secrets management, and controlled service exposure protect sensitive configuration values. Regular monitoring, logging, and auditing mechanisms detect anomalous activity in real time. The system adopts a multi-layered security approach combining application-level, container-level, and infrastructure-level security measures to ensure data integrity, confidentiality, and availability.

7.2 Performance Aspects

Performance is a key requirement ensuring fast response times and seamless user experience as usage scales. Kubernetes-based horizontal scaling allows the system to increase pods based on real-time demand without performance degradation. Semantic caching via ChromaDB significantly reduces re-invocation of expensive AI model calls for similar queries, lowering both latency and API costs. Load balancing within Kubernetes distributes traffic evenly across service instances.

The multi-agent system executes research and writing tasks concurrently where possible, reducing total report generation time. Automated CI/CD pipelines maintain performance standards by continuously testing updates and preventing performance regressions from reaching production. Monitoring tools track response time, CPU usage, memory utilization, and AI API latency, enabling proactive tuning and issue resolution.

REFERENCES

- Docker Official Documentation for containerization and environment consistency.
- Kubernetes Official Documentation for container orchestration, scaling, and deployment strategies.
- OpenAI API Documentation for language model and image generation integration.
- CrewAI Framework Documentation for multi-agent system design and implementation.
- Streamlit Official Documentation for frontend interface development.
- ChromaDB Documentation for vector database, embedding storage, and semantic search.
- GitHub Actions Documentation for CI/CD workflow automation and continuous integration practices.
- Git Official Documentation for version control, collaboration, and pipeline integration.
- Research papers and articles on Generative AI, Agentic AI systems, and AI-driven manufacturing automation.
- Industry best practices and guidelines for microservices architecture, security, and scalable AI system design.